

Lab 5 Explainer: From Bag-of-Words to Word2Vec (CBOW & Skipgram)

A walkthrough of Lab_5_Tutorial_CBOW.ipynb

Part A. Bag-of-Words

Before dense embeddings, the simplest way to turn text into numbers is Bag-of-Words (BoW): a document becomes a vector of word counts over a fixed vocabulary. Word order and grammar are thrown away entirely — all that survives is which words appeared, and how many times.

```
BoW(doc)[i] = count of vocab[i] in doc
```

Even though BoW throws away grammar, it can still say “these two documents use similar words.” The “king” sentence and the “queen” sentence in the tutorial share every word except one, so they get a clearly higher cosine similarity than either has with the unrelated “wizard” sentence — BoW is a genuinely useful, simple technique for comparing whole documents.

Where Bag-of-Words breaks down

Now isolate just the words “king” and “queen” as one-hot vectors — each word its own independent dimension. Their cosine similarity is exactly 0: completely orthogonal, as unrelated geometrically as “king” and “spaceship.” This is the whole point of Part A: BoW / one-hot vectors have no mechanism to encode that two words are related in meaning. That gap is exactly what Part B and Part C exist to close.

Question: If BoW can compare documents fine, why do we need embeddings at all?

Answer: BoW similarity only works when documents literally share vocabulary. It cannot tell you that 'the king ruled wisely' and 'the monarch governed wisely' are about the same thing, because 'king' and 'monarch' are unrelated dimensions to it. Dense embeddings fix exactly this.

Part B. Playing with pretrained word vectors

Training genuinely good embeddings needs billions of words of text — far more than we can train on in this course. GloVe (loaded via gensim) gives us a look at what well-trained dense vectors can do, before we train our own smaller version in Part C.

```
word_vectors = gensim.downloader.load('glove-wiki-gigaword-50')
```

Similarity() gives a graded, meaningful score, not just 0 or 1 like Part A's one-hot vectors — king/queen comes out clearly higher than king/cabbage.

The analogy: king - man + woman ≈ queen

most_similar(positive=['king', 'woman'], negative=['man']) computes king - man + woman and returns the closest words to that resulting vector. “Queen” shows up at or near the top — and this isn't a lookup table of royalty. It falls out purely from vector arithmetic on directions the model learned from how words co-occur in text.

```
word_vectors.most_similar(positive=['king', 'woman'], negative=['man'], topn=5)
# -> queen near the top
```

Visualising the analogy

If king - man + woman ≈ queen, then rearranging: king - man ≈ queen - woman. So the arrow from man to king should point in roughly the same direction and length as the arrow from woman to queen. Reducing the four vectors to 2D with PCA and drawing both arrows lets us see this directly. The two arrows come out visibly close to parallel — that closeness is the analogy, described with a picture instead of subtraction and addition.

```
king - man ≈ queen - woman
=> arrow(man→king) roughly parallel to arrow(woman→queen)
```

Question: Are the arrows ever perfectly parallel?

Answer: No, and that's worth saying explicitly: PCA is compressing a 50-dimensional space down to 2 dimensions for us to look at, so some of the relationship is necessarily lost. The cosine-similarity number in the notebook (typically well above 0.9 for this example) is the more precise way to state 'how parallel'.

Question: Why does this work for king/queen but might not work perfectly for every analogy students try?

Answer: The relationship has to be a common, consistently-used pattern in the training text for the model to learn a clean direction for it. Rare or unusual relationships, or words with multiple meanings, tend to produce messier results — a good segue into why Part C's much smaller, from-scratch model won't match GloVe's cleanliness.

Part C, 1-2. From raw text to tokens, and the vocabulary

Word2vec starts as a bookkeeping problem before any neural network is involved: every word needs a fixed integer id, because neural networks operate on numbers, not strings. In the toy sentence 'The boy who lived came to die, but the boy who lived came back,' lowercasing plus stripping punctuation collapses 'The' and 'the' into the same token. The vocabulary is just the sorted set of unique tokens, and `word_to_int` / `int_to_word` are simple inverse lookup tables.

```
vocab = sorted(set(tokens))
word_to_int[w] = index of w in vocab
int_to_word[i] = the word at that index
```

Question: Why do we sort the vocabulary?

Answer: It isn't required for correctness — any consistent word→index mapping works — but sorting makes the mapping deterministic and reproducible across runs, which makes debugging and grading much easier.

Part C, 3. Generating (context, target) pairs

Generating (context, target) pairs is the step that turns unstructured text into a supervised learning problem. For every position in the token list, the word at that position is the label to predict (the target), and the words within `window_size` on each side are the input features (the context). Note that we do NOT pad at the edges of the text — the first and last few words simply get a shorter context list.

```
tokens: [the, boy, who, lived, came, to, die]
i = 2, target = who
context = tokens[0:2] + tokens[3:5] = [the, boy, lived, came]
```

Question: What happens with `window_size = 2` at the very first word?

Answer: There is nothing to its left, so the context is just the two words to its right — left slicing naturally returns an empty list, so no special-case code is needed if you use Python slicing carefully.

Part C, 4. The CBOW model & forward pass

One-hot context words go in, get looked up in W_1 (this lookup plus averaging IS the “hidden layer” — there's no non-linearity here, which is what makes CBOW simple), the averaged vector h is multiplied by W_2 to produce a raw score for every word in the vocabulary, and softmax turns those scores into a probability distribution. The whole point of training is to make the model put high probability on the actual target word.

```
h = average of W1 rows for the context word indices
u = h . W2                                (scores over the vocabulary)
y_hat = softmax(u)                        (probabilities, sum to 1)
```

Numerical stability of softmax

If any score in u is even moderately large, $\exp(u)$ overflows to `inf`, and `inf / inf` becomes `nan` — try computing a naive (non-shifted) softmax on large scores and you'll see this happen. The fix is a well-known algebraic identity: subtracting the max score from every entry before exponentiating does not change the softmax output (it cancels in the ratio), but keeps every exponent ≤ 0 , so $\exp$ never overflows.

```
softmax(u)_j = exp(u_j - max(u)) / sum_k exp(u_k - max(u))
```

Question: Why does subtracting the max not change the answer?

Answer: $\text{softmax}(u + c) = \text{softmax}(u)$ for any constant c added to every entry — the c 's cancel between numerator and denominator once you factor $\exp(c)$ out. Choosing $c = -\max(u)$ is just the specific choice that keeps the numbers small.

Part C, 5-6. Cross-entropy loss & the backward pass derivation

Loss is $-\log(\hat{y}_{\text{at the true target index}})$. If the model is very confident and correct, $\hat{y}_{\text{target}}$ is close to 1 and the loss is close to 0. If the model puts almost no probability on the correct word, the loss blows up. This is the same cross-entropy loss from any classification setting; here the 'classes' happen to be every word in the vocabulary.

```
L = -log(y_hat[target_idx])
```

For the backward pass, it helps to work through the three-step chain rule by hand rather than jumping straight to code — this is the part people get wrong most often.

- Step 1: Because softmax and cross-entropy are paired, the gradient of the loss w.r.t. the pre-softmax scores u collapses to a very simple form: $e = \hat{y} - y_{\text{true}}$ (the one-hot target vector). This is worth deriving on the board once so students trust it rather than memorise it.
- Step 2: Since $u = h \cdot W_2$, the gradient for W_2 is the outer product of h and e .
- Step 3: The gradient for W_1 only touches the rows corresponding to context words, because those are the only rows that contributed to h . Since h was an average, the gradient w.r.t. h gets split equally across all context word rows.

```
e = y_hat - y_true
dW2 = outer(h, e)
dh = W2 . e
dW1[c_k] += dh / len(context)    for every context index c_k
(all other rows of dW1 stay exactly zero)
```

Question: Why is it OK that most of dW_1 is zero after one example?

Answer: Each training example only involves a handful of words, so it should only update the embeddings of those words. Over many examples across the whole corpus, every word eventually appears in enough contexts to get a well-trained embedding.

Part C, 7-8. One SGD step, then the full training loop

Before trusting a multi-epoch loop, run the cheap sanity check shown in the tutorial: take exactly one gradient step on one example and confirm the loss on that same example went down. If it doesn't, there's a bug in forward or backward — find it before running a slow training loop on the real corpus.

```
W_new = W_old - learning_rate * dW
```

Running the training loop on the toy sentence, the loss drops sharply because the toy corpus is tiny and easy to memorise. On the real Harry Potter corpus, the loss will fall more slowly and will not reach zero — that's expected, not a bug.

Question: My loss isn't decreasing at all in the assignment, what should I check first?

Answer: Re-run the tutorial's one-step sanity check with your own forward/backward code substituted in. If loss doesn't decrease on a single tiny example after one update, the bug is in forward or backward, not in your training loop or your corpus size.

Part C, 9. Evaluation: cosine similarity & nearest neighbours

Once training is done, W_1 IS the embedding table — there's nothing further to extract. Cosine similarity compares the direction of two vectors, ignoring their length, which is why it's preferred over raw dot product or Euclidean distance for embeddings (word frequency can inflate vector length without changing meaning).

```
cos(v1, v2) = (v1 . v2) / (||v1|| * ||v2||)
```

Question: Why not just compare raw embedding vectors with Euclidean distance?

Answer: Two vectors can point in almost the same direction (i.e. represent similar words) but have very different magnitudes, especially since frequent words tend to get larger embedding norms during training. Cosine similarity factors out magnitude and focuses purely on direction.

Part D. Skip-gram — the mirror image of CBOW (bonus)

Think of Skip-gram as “same machinery, flipped direction.” CBOW predicts one target from several averaged context words; Skip-gram predicts several individual context words from one target. Its forward pass is actually simpler than CBOW's — there's only one input word, so there's no averaging step at all. h is just that word's row of W_1 , full stop.

```
CBOW:      h = average of context rows of W1
Skip-gram: h = W1[input_idx]    (no averaging)
```

For pair generation, note the shape difference: CBOW makes one training pair per position (one context list, one target). Skip-gram makes one pair per (position, context word) — a target with 4 context words becomes 4 separate training pairs.

Backward pass: the gradient isn't split

In CBOW, the gradient reaching h got divided by the number of context words and spread across all of them, because h was their average. In Skip-gram, h came from a single word, so that word's row gets the entire gradient, undivided. This is a small, clean modification of the CBOW backward pass derivation.

```
CBOW:      dW1[c_k] += dh / len(context)    for each context word c_k
Skip-gram: dW1[input_idx] = dh              (whole gradient, no division)
```

Question: Why does Skip-gram need more training pairs to cover the same text?

Answer: Because every context word around a target becomes its own separate example, instead of being bundled into one averaged example like in CBOW. More pairs from the same text, but each one is a 'smaller', more specific piece of supervision.

Comparing the loss curves

This is the most important thing to know going in: when you plot CBOW's loss against Skip-gram's on the same toy sentence, Skip-gram's loss plateaus noticeably higher and CBOW's falls toward zero. This is NOT a bug. A frequent word like 'the' produces the identical input vector every time it appears, but Skip-gram has to predict several different correct context words from that one identical input — it mathematically cannot beat the entropy of that spread. CBOW doesn't have this problem because its input (the averaged context) varies slightly from occurrence to occurrence.

Question: If Skip-gram's loss looks worse, why would anyone use it?

Answer: This exact one-to-many tension is much less of a problem for rare words, since a rare word appears in far fewer distinct contexts to begin with. On a large, varied corpus, Skip-gram tends to give rare words more, cleaner individual gradient updates than CBOW, where a rare word might just get averaged in alongside more frequent neighbours and diluted. This is exactly what Analysis Question Q4 in the assignment asks you to investigate on the real Harry Potter corpus, after predicting it yourself beforehand.

Wrap-up. Mapping the tutorial onto the assignment

The pipeline below maps each stage of the tutorial explicitly to the assignment's task numbers. The formulas and shapes are identical between the tutorial and the assignment — the only things that change are vocabulary size, embedding dimension, and number of training pairs/epochs. If the pure-Python training loop is too slow on the full corpus, train on a subset of tokens (e.g. the first 20k-50k) and say so explicitly in your submission — the assignment explicitly allows this.

- Task A1 (Build BoW, cosine similarity) $\leftrightarrow$ tutorial Part A
- Task A2 (Why BoW falls short) $\leftrightarrow$ tutorial Part A, 'Where it breaks down'
- Task B1 (Similarity with real vectors) $\leftrightarrow$ tutorial Part B, similarity section
- Task B2 (king - man + woman analogy) $\leftrightarrow$ tutorial Part B, 'The main event'

- Task B3 (Your own analogies) $\leftrightarrow$ tutorial Part B, 'Try it yourself'
- Task B4 (Parallelogram visualisation) $\leftrightarrow$ tutorial Part B, 'Making it visual'
- Task 1 (Preprocessing) $\leftrightarrow$ tutorial Part C, Sections 1-2
- Task 2 (CBOW pairs) $\leftrightarrow$ tutorial Part C, Section 3
- Task 3 (Forward pass, stable softmax) $\leftrightarrow$ tutorial Part C, Section 4
- Task 4 (Loss, backward pass) $\leftrightarrow$ tutorial Part C, Sections 5-6
- Task 5 (Training loop + loss plot) $\leftrightarrow$ tutorial Part C, Sections 7-8
- Task 6 (Cosine similarity, nearest neighbours, analogy) $\leftrightarrow$ tutorial Part C, Section 9
- Task 7 (Bonus: PCA visualisation of your own embeddings) — a natural extension of Part C, Section 9, reusing the same PCA technique from Task B4 on your own trained vectors instead of GloVe's
- Task D1-D4 (Skip-gram: pairs, forward, backward, train & compare) $\leftrightarrow$ tutorial Part D

A reminder: Part A, Part C, and Part D use no ML libraries (no `torch.nn`, no `gensim`, no `sklearn` except the Part C bonus). Part B is the one place a library is required — `gensim` to load pretrained vectors, `sklearn`'s PCA for the parallelogram plot. The whole point of this lab is to build your intuition for what those libraries are doing under the hood, by seeing the payoff first (Part B) and then building it yourself (Parts C and D).